← 回學習地圖

Yahboom MicroROS-Pi5 實車演練 Part 1：指令與輸出紀錄

2026-10-27 在真車上（車子架空）由「YB 教練」逐段操作的完整紀錄：每個指令在哪裡執行、實際下的指令、車子回傳的完整輸出，以及教練當下的解說。精華影片播放清單：實車演練 Part 1（精華版）。

連線方式：筆電經 USB → 手機熱點轉接到車上 Pi 5（10.57.201.1），用 `ycar.py` 對 Pi 主機或 ROS 2 容器下指令。顏色追蹤（M3）這次跳過；鍵盤遙控因官方程式初速超過安全上限而跳過。

1. 第 1 段｜連線與環境（B2＋W3）
2. 第 2 段｜micro-ROS agent 與 Docker 容器（B3）
3. 第 3 段｜讀懂車的話題：電池、光達、輪子、雲台、蜂鳴器（B5）
4. 第 4 段｜手把遙控與搶方向盤（B4）
5. 插曲｜相機 USB 斷線排查（硬體除錯）
6. 第 5 段｜OpenCV 與影像話題（M1）
7. 第 6 段｜MediaPipe 手部與臉部偵測（M2）
8. 第 7 段｜自己寫的避障節點上真車（M5）

第 1 段｜連線與環境 B2＋W3

▶ 看這段的精華影片（YouTube 私人影片，需登入擁有者帳號）

▼ 教練解說（7 句）

1. 今天我們在真車上演練。筆電透過手機熱點連到車子，螢幕三是用 VNC 看到的 Pi 5 桌面，左邊是我用 SSH 下指令。第一段，先確認連線和環境。
2. 先問車子：你叫什麼名字、IP 是多少、是哪一塊板子。
3. 車子在熱點上的 IP 是 10.57.201.1，另一個 172.17 開頭的是 Docker 內部網路。帳號是 pi，板子是樹莓派 5，已經開機四十一分鐘。接著看三個硬體裝置在不在。
4. 序列埠和手把接收器都在，但相機 video0 沒看到，我查一下相機現在叫什麼名字。
5. 找到原因了：相機在晚上八點二十三分 USB 斷線，現在系統裡沒有相機，video19 以後那些是樹莓派內建的影像處理裝置，不是鏡頭。相機線可能鬆了，這段結束後再請你檢查。我們先繼續看容器。
6. 有兩個容器在跑：一個是 ROS 2 工作室，所有 ROS 程式都在裡面；另一個是 micro-ROS 翻譯官，負責跟車底的 ESP32 溝通。接著看映像檔和啟動腳本。
7. 映像檔版本是 4.1.2，就是現在在跑的那個；4.1.0 是舊版沒在用。啟動腳本有個小筆誤：X11 那一行寫成斜線 tem，應該是 tmp，不過我們的視窗是另外處理的，不影響今天。還有一點：容器是用掛載 video0 的方式拿相機，相機重新插上後，可能要確認容器裡看不看得到。第一段完成。

指令與輸出（7 個）

Pi 5 主機 Speak then query Pi hostname, IP, model

```
$ hostname -I; whoami; cat /proc/device-tree/model
```

```
10.57.201.1 172.17.0.1 2402:7500:a2d:b982:1e8:506:109d:558a
pi
Raspberry Pi 5 Model B Rev 1.0
 20:46:00 up 41 min,  2 users,  load average: 0.20, 0.16, 0.06
```

Pi 5 主機 Explain output and list device files

```
$ ls -l /dev/ttyUSB0 /dev/video0 /dev/input/js0
```

```
Exit code 2
```

```
ls: cannot access '/dev/video0': No such file or directory
crw-rw----+ 1 root input    13, 0 Oct 15 15:17 /dev/input/js0
crw-rw----  1 root dialout 188, 0 Oct 26 20:46 /dev/ttyUSB0
```

Pi 5 主機 Look for camera device nodes and USB list

```
$ ls -l /dev/video*
```

```
crw-rw----+ 1 root video 81, 18 Oct 15 15:17 /dev/video19
crw-rw----+ 1 root video 81,  0 Oct 15 15:17 /dev/video20
crw-rw----+ 1 root video 81,  1 Oct 15 15:17 /dev/video21
crw-rw----+ 1 root video 81,  2 Oct 15 15:17 /dev/video22
crw-rw----+ 1 root video 81,  3 Oct 15 15:17 /dev/video23
crw-rw----+ 1 root video 81,  4 Oct 15 15:17 /dev/video24
crw-rw----+ 1 root video 81,  5 Oct 15 15:17 /dev/video25
crw-rw----+ 1 root video 81,  6 Oct 15 15:17 /dev/video26
crw-rw----+ 1 root video 81,  7 Oct 15 15:17 /dev/video27
crw-rw----+ 1 root video 81,  8 Oct 15 15:17 /dev/video28
crw-rw----+ 1 root video 81,  9 Oct 15 15:17 /dev/video29
crw-rw----+ 1 root video 81, 10 Oct 15 15:17 /dev/video30
crw-rw----+ 1 root video 81, 11 Oct 15 15:17 /dev/video31
crw-rw----+ 1 root video 81, 12 Oct 15 15:17 /dev/video32
crw-rw----+ 1 root video 81, 13 Oct 15 15:17 /dev/video33
crw-rw----+ 1 root video 81, 14 Oct 15 15:17 /dev/video34
crw-rw----+ 1 root video 81, 15 Oct 15 15:17 /dev/video35
crw-rw----+ 1 root video 81, 16 Oct 15 15:17 /dev/video36
crw-rw----+ 1 root video 81, 17 Oct 15 15:17 /dev/video37
---
ls: cannot access '/dev/v4l/by-id/': No such file or directory
---
Bus 004 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 003 Device 003: ID 10c4:ea60 Silicon Labs CP210x UART Bridge
Bus 003 Device 002: ID 0079:181c DragonRise Inc. Controller
Bus 003 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
```

Pi 5 主機 Check kernel log for camera USB events

```
$ dmesg 2>/dev/null | grep -iE 'uvc|camera|usb [0-9-]+: (new|USB disconnect)' | tail -15
```

```
[    0.669196] usb 1-1: new high-speed USB device number 2 using xhci-hcd
[    0.669262] usb 3-1: new full-speed USB device number 2 using xhci-hcd
[    0.843412] usb 3-1: New USB device found, idVendor=0079, idProduct=181c, bcdDevice=10.
03
[    0.843418] usb 3-1: New USB device strings: Mfr=0, Product=2, SerialNumber=0
[    0.891889] usb 1-1: New USB device found, idVendor=0c45, idProduct=6368, bcdDevice=27.
02
```

```
[    0.891894] usb 1-1: New USB device strings: Mfr=2, Product=1, SerialNumber=0
[    0.891896] usb 1-1: Product: USB 2.0 Camera
[    1.113161] usb 3-2: new full-speed USB device number 3 using xhci-hcd
[    1.310435] usb 3-2: New USB device found, idVendor=10c4, idProduct=ea60, bcdDevice= 1.
00
[    1.310438] usb 3-2: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[    3.957227] usb 1-1: Found UVC 1.00 device USB 2.0 Camera (0c45:6368)
[    4.060289] usbcore: registered new interface driver uvcvideo
[ 1116.906768] usb 1-1: USB disconnect, device number 2
Oct 15 15:17:02 raspberrypi kernel: usb 1-1: Found UVC 1.00 device USB 2.0 Camera (0c45:63
68)
Oct 15 15:17:02 raspberrypi kernel: usbcore: registered new interface driver uvcvideo
Oct 26 20:23:37 raspberrypi kernel: usb 1-1: USB disconnect, device number 2
```

Pi 5 主機 Narrate camera issue and list containers

```
$ docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'
```

```
NAMES              IMAGE                                STATUS
wizardly_mclaren   yahboomtechnology/ros-humble:4.1.2   Up 17 hours
nice_northcutt     microros/micro-ros-agent:humble      Up 17 hours
```

Pi 5 主機 Narrate and show docker images and startup script

```
$ docker images
```

Pi 5 主機 Narrate and show docker images and startup script

```
$ cat ~/ros2_humble.sh
```

```
REPOSITORY                     TAG       IMAGE ID       CREATED       SIZE
yahboomtechnology/ros-humble   4.1.2     9bde3b769007   2 years ago   13.6GB
yahboomtechnology/ros-humble   4.1.0     caecba031263   2 years ago   13.6GB
microros/micro-ros-agent       humble    5a1c5ccb63d4   3 years ago   404MB
=====
#!/bin/bash
xhost +
docker run -it \
--privileged=true \
--net=host \
--env="DISPLAY" \
--env="QT_X11_NO_MITSHM=1" \
-v /tem/.X11-unix:/tmp/.X11-unix \
--security-opt apparmor:unconfined \
-v /dev/input:/dev/input \
```

```
-v /dev/video0:/dev/video0 \
yahboomtechnology/ros-humble:4.1.2 /bin/bash
```

第 2 段｜micro-ROS agent 與 Docker 容器 B3

▶ 看這段的精華影片（YouTube 私人影片，需登入擁有者帳號）

▼ 教練解說（5 句）

1. 第二段，看 micro-ROS agent 和容器。agent 是翻譯官，把車底 ESP32 的訊息翻成 ROS 2 話題。先看它的啟動腳本。
2. agent 走序列埠 ttyUSB0，速度 921600。注意它把整個 dev 資料夾掛進去，所以裝置重插也看得到。再看 agent 最近的紀錄。
3. 紀錄裡一直出現 topic created、subscriber created，代表 ESP32 已經連上，正在建立話題。接著進 ROS 容器，看網域編號和有哪些節點。
4. 網域是 20。YB_Car_Node 就是車底 ESP32，我叫它小腦；ycar_monitor 是右邊儀表板在用的節點。看看小腦訂閱什麼、發布什麼。
5. 小腦聽四個話題：cmd_vel 控制輪子、servo s1 和 s2 控制雲台左右和上下、beep 控制蜂鳴器。它發布四個話題：電池、IMU 姿態、輪子里程計、還有光達掃描。也就是說，我們只要對這幾個話題說話，就能控制整台車。第二段完成。

指令與輸出（5 個）

Pi 5 主機 Narrate and show agent start script

```
$ cat ~/start_agent_rpi5.sh | grep -v '^#'
```

```
docker run -it --rm -v /dev:/dev -v /dev/shm:/dev/shm --privileged --net=host microros/mic
ro-ros-agent:humble serial --dev /dev/ttyUSB0 -b 921600
```

Pi 5 主機 Narrate and show agent logs

```
$ docker logs --tail 5 $(docker ps -q --filter ancestor=microros/micro-ros-agent:humble)
```

```
[1792986519.684828] info     | ProxyClient.cpp    | create_subscriber        | subscriber
created     | client_key: 0x5370504A, subscriber_id: 0x002(4), participant_id: 0x000(1)

[1792986519.689539] info     | ProxyClient.cpp    | create_datareader        | datareader
created     | client_key: 0x5370504A, datareader_id: 0x002(6), subscriber_id: 0x002(4)

[1792986519.694855] info     | ProxyClient.cpp    | create_topic             | topic creat
ed          | client_key: 0x5370504A, topic_id: 0x007(2), participant_id: 0x000(1)
```

```
[1792986519.698860] info     | ProxyClient.cpp    | create_subscriber        | subscriber
created     | client_key: 0x5370504A, subscriber_id: 0x003(4), participant_id: 0x000(1)

[1792986519.706062] info     | ProxyClient.cpp    | create_datareader        | datareader
created     | client_key: 0x5370504A, datareader_id: 0x003(6), subscriber_id: 0x003(4)
```

車上 ROS 2 容器 Narrate and list ROS nodes

```
$ echo DOMAIN=$ROS_DOMAIN_ID; ros2 node list
```

```
DOMAIN=20
/YB_Car_Node
/rqt_gui_cpp_node_2391
/ycar_monitor
```

車上 ROS 2 容器 Show YB_Car_Node info and check stray rqt

```
$ ros2 node info /YB_Car_Node
```

車上 ROS 2 容器 Show YB_Car_Node info and check stray rqt

```
$ ps -eo pid,etime,cmd | grep -i [r]qt
```

```
/YB_Car_Node
  Subscribers:
    /beep: std_msgs/msg/UInt16
    /cmd_vel: geometry_msgs/msg/Twist
    /servo_s1: std_msgs/msg/Int32
    /servo_s2: std_msgs/msg/Int32
  Publishers:
    /battery: std_msgs/msg/UInt16
    /imu: sensor_msgs/msg/Imu
    /odom_raw: nav_msgs/msg/Odometry
    /scan: sensor_msgs/msg/LaserScan
  Service Servers:

  Service Clients:

  Action Servers:

  Action Clients:

=====
   2391       07:09 python3 /opt/ros/humble/lib/rqt_image_view/rqt_image_view
```

第 3 段｜讀懂車的話題：電池、光達、輪子、雲台、蜂鳴器 B5

▶ 看這段的精華影片（YouTube 私人影片，需登入擁有者帳號）

▼ 教練解說（11 句）

1. 第三段，讀懂車子的話題。先列出所有話題和它們的型別。
2. 除了小腦那八個話題，還有一個 image raw，那是之前開的影像視窗在等畫面。不過相機還是沒插回來，主機看不到，所以相機畫面這步先跳過，雲台就請你直接看實體鏡頭轉動。先讀電池。
3. 電池回報 74，單位是零點一伏，所以大約 7.4 伏，兩顆鋰電池串聯，還算夠用，但偏低了，等一下輪子轉的時候注意一下。接著量光達掃描的頻率。
4. 再量 IMU 姿態感測器。
5. 光達每秒大約 12.5 圈，IMU 每秒大約 18 筆。最後那段紅字是 timeout 時間到把它關掉，正常的。接下來讓輪子轉：每秒 0.1 公尺前進三秒，請看右邊儀表板的輪子狀態。
6. 停了。剛剛輪子應該轉了三秒。接著原地左轉，每秒 0.5 弧度，一樣三秒。
7. 接著是雲台。相機沒接上，請直接看車上的鏡頭支架。為了安全，每次只轉五度，先往左轉到 60 度，再回到中間。
8. 再往右轉到負 60 度，再回中間。servo s1 正的是左、負的是右。
9. 換上下，servo s2。先抬頭到 15 度，再低頭到負 45 度，最後回正。
10. 雲台回正了。最後是蜂鳴器，接下來嗶一聲，很短。
11. 嗶，50 代表響 0.05 秒。這段我們讀了電池、光達、IMU，也讓輪子、雲台、蜂鳴器都動過一次。這段沒開相機程式，所以不用關。第三段完成。

指令與輸出（17 個）

車上 ROS 2 容器 List topics and check camera on host and container

```
$ ros2 topic list -t
```

Pi 5 主機 List topics and check camera on host and container

```
$ ls -l /dev/video0
```

車上 ROS 2 容器 List topics and check camera on host and container

```
$ ls -l /dev/video0
```

```
/battery [std_msgs/msg/UInt16]
/beep [std_msgs/msg/UInt16]
/cmd_vel [geometry_msgs/msg/Twist]
/image_raw [sensor_msgs/msg/Image]
/imu [sensor_msgs/msg/Imu]
/odom_raw [nav_msgs/msg/Odometry]
/parameter_events [rcl_interfaces/msg/ParameterEvent]
/rosout [rcl_interfaces/msg/Log]
/scan [sensor_msgs/msg/LaserScan]
/servo_s1 [std_msgs/msg/Int32]
/servo_s2 [std_msgs/msg/Int32]
=====
ls: cannot access '/dev/video0': No such file or directory
主機：沒看到相機
c---------+ 0 root root 81, 19 Oct 15 14:17 /dev/video0
```

車上 ROS 2 容器 Narrate camera skip and read battery

```
$ ros2 topic echo /battery --once
```

```
data: 74
---
```

車上 ROS 2 容器 Measure scan and imu topic rates

```
$ timeout 6 ros2 topic hz /scan
```

車上 ROS 2 容器 Measure scan and imu topic rates

```
$ timeout 6 ros2 topic hz /imu
```

```
Exit code 124


average rate: 12.501
        min: 0.061s max: 0.089s std dev: 0.00765s window: 14
average rate: 12.489
        min: 0.058s max: 0.095s std dev: 0.00824s window: 27
average rate: 12.473
        min: 0.058s max: 0.096s std dev: 0.00848s window: 40
average rate: 12.496
        min: 0.058s max: 0.096s std dev: 0.00849s window: 53
Traceback (most recent call last):
  File "/opt/ros/humble/bin/ros2", line 33, in <module>
    sys.exit(load_entry_point('ros2cli==0.18.7', 'console_scripts', 'ros2')())
  File "/opt/ros/humble/lib/python3.10/site-packages/ros2cli/cli.py", line 89, in main
```

```
    rc = extension.main(parser=parser, args=args)
  File "/opt/ros/humble/lib/python3.10/site-packages/ros2topic/command/topic.py", line 4
1, in main
    return extension.main(args=args)
  File "/opt/ros/humble/lib/python3.10/site-packages/ros2topic/verb/hz.py", line 89, in
main
    return main(args)
  File "/opt/ros/humble/lib/python3.10/site-packages/ros2topic/verb/hz.py", line 104, in
main
    _rostopic_hz(node.node, topic, window_size=args.window_size, filter_expr=filter_exp
r,
  File "/opt/ros/humble/lib/python3.10/site-packages/ros2topic/verb/hz.py", line 273, in
_rostopic_hz
    rclpy.spin_once(node)
  File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/__init__.py", line 202,
in spin_once
    executor.spin_once(timeout_sec=timeout_sec)
  File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/executors.py", line 73
9, in spin_once
    self._spin_once_impl(timeout_sec)
  File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/executors.py", line 72
8, in _spin_once_impl
    handler, entity, node = self.wait_for_ready_callbacks(timeout_sec=timeout_sec)
  File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/executors.py", line 71
1, in wait_for_ready_callbacks
    return next(self._cb_iter)
  File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/executors.py", line 61
2, in _wait_for_ready_callbacks
    raise ExternalShutdownException()
rclpy.executors.ExternalShutdownException


average rate: 18.198
        min: 0.011s max: 0.082s std dev: 0.02269s window: 20
average rate: 18.130
        min: 0.010s max: 0.084s std dev: 0.02531s window: 39
average rate: 18.612
        min: 0.010s max: 0.084s std dev: 0.02523s window: 59
average rate: 18.439
        min: 0.008s max: 0.084s std dev: 0.02541s window: 78
…（以下省略 24 行）
```

車上 ROS 2 容器 Drive forward 0.1 m/s for 3 s then stop

```
$ timeout 3 ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist '{linear: {x: 0.1}}'
```

停車 Drive forward 0.1 m/s for 3 s then stop

```
$ ycar.py stopcar
```

```
/cmd_vel 訂閱者數量：2
已送出零速度 /cmd_vel 第 1 次
已送出零速度 /cmd_vel 第 2 次
已送出零速度 /cmd_vel 第 3 次
已送出 /servo_s1=0、/servo_s2=0、/beep=0
```

車上 ROS 2 容器 Rotate left 0.5 rad/s for 3 s then stop

```
$ timeout 3 ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist '{angular: {z: 0.5}}'
```

停車 Rotate left 0.5 rad/s for 3 s then stop

```
$ ycar.py stopcar
```

```
/cmd_vel 訂閱者數量：2
已送出零速度 /cmd_vel 第 1 次
已送出零速度 /cmd_vel 第 2 次
已送出零速度 /cmd_vel 第 3 次
已送出 /servo_s1=0、/servo_s2=0、/beep=0
```

車上 ROS 2 容器 Pan servo left to 60 deg in 5-deg steps and back

```
$ for a in $(seq 5 5 60) $(seq 55 -5 0); do ros2 topic pub --once /servo_s1 std_msgs/msg/Int32 "{data: $a}" >/dev/null
```

```
5 10 15 20 25 30 35 40 45 50 55 60 55 50 45 40 35 30 25 20 15 10 5 0
```

車上 ROS 2 容器 Pan servo right to -60 deg and back

```
$ for a in $(seq -5 -5 -60) $(seq -55 5 0); do ros2 topic pub --once /servo_s1 std_msgs/msg/Int32 "{data: $a}" >/dev/null
```

```
-5 -10 -15 -20 -25 -30 -35 -40 -45 -50 -55 -60 -55 -50 -45 -40 -35 -30 -25 -20 -15 -10 -5 0
```

車上 ROS 2 容器 Tilt servo up 15, down -45, back to 0

```
$ for a in 5 10 15 10 5 0 $(seq -5 -5 -45) $(seq -40 5 0); do ros2 topic pub --once /servo_s2 std_msgs/msg/Int32 "{data: $a}" >/dev/null
```

```
5 10 15 10 5 0 -5 -10 -15 -20 -25 -30 -35 -40 -45 -40 -35 -30 -25 -20 -15 -10 -5 0
```

車上 ROS 2 容器 Short beep 50 ms

```
$ ros2 topic pub --once /beep std_msgs/msg/UInt16 '{data: 50}'
```

```
publisher: beginning loop
publishing #1: std_msgs.msg.UInt16(data=50)
```

狀態總覽 Check status and narrate segment 3 end

```
$ ycar.py status
```

```
/YB_Car_Node
/rqt_gui_cpp_node_2391
/ycar_monitor
== /battery（一筆）
data: 74
---
== /scan 頻率（6 秒）
average rate: 12.356
        min: 0.074s max: 0.089s std dev: 0.00463s window: 14
average rate: 12.494
        min: 0.061s max: 0.098s std dev: 0.00863s window: 27
average rate: 12.468
        min: 0.057s max: 0.098s std dev: 0.00851s window: 40
average rate: 12.502
        min: 0.057s max: 0.098s std dev: 0.00946s window: 53
```

車上 ROS 2 容器 Inspect joystick node speed limits and button map

```
$ f=$(find /root/yahboomcar_ws/src/yahboomcar_ctrl -name 'yahboom_joy*.py' | head -1)
```

```
/root/yahboomcar_ws/src/yahboomcar_ctrl/yahboomcar_ctrl/yahboom_joy.py
17:from std_msgs.msg import Int32, Bool,UInt16

35:             self.s1_init_angle = Int32()

37:             self.s2_init_angle = Int32()

44:             self.pub_Servo1 = self.create_publisher(Int32,"servo_s1" , 10)

45:             self.pub_Servo2 = self.create_publisher(Int32,"servo_s2" , 10)

52:             self.declare_parameter('xspeed_limit',1.0)
```

```
53:             self.declare_parameter('yspeed_limit',1.0)

54:             self.declare_parameter('angular_speed_limit',5.0)

55:             self.xspeed_limit = self.get_parameter('xspeed_limit').get_parameter_val
ue().double_value

56:             self.yspeed_limit = self.get_parameter('yspeed_limit').get_parameter_val
ue().double_value

57:             self.angular_speed_limit = self.get_parameter('angular_speed_limit').get
_parameter_value().double_value

69:             self.servo_angle = Int32()

71:                     self.servo_angle.data =  (angle & 0xff)<<16|(91 )

74:                     self.servo_angle.data =  (91 )<<16|(angle & 0xff)

75:             print(self.servo_angle.data)

76:             self.pub_Servo.publish(self.servo_angle)

80:             if joy_data.buttons[7] == 1: self.cancel_nav()

82:             if joy_data.buttons[11] == 1:

91:             xlinear_speed = self.filter_data(joy_data.axes[1]) * self.xspeed_limit *
self.linear_Gear

92:         #ylinear_speed = self.filter_data(joy_data.axes[2]) * self.yspeed_limit * sel
f.linear_Gear

93:             #ylinear_speed = self.filter_data(joy_data.axes[0]) * self.yspeed_limit
* self.linear_Gear

94:             angular_speed = self.filter_data(joy_data.axes[2]) * self.angular_speed_
limit * self.angular_Gear

…（以下省略 100 行）
```

車上 ROS 2 容器 Read joy node init and servo button logic

```
$ sed -n '28,66p;100,170p' /root/yahboomcar_ws/src/yahboomcar_ctrl/yahboomcar_ctrl/yahboom
_joy.py
```

```
self.angular_Gear = 1.0 / 4

                self.Servo_leftX = 0
```

```
                self.Servo_rightB = 0

                self.Servo_downA = 0

                self.Servo_upY = 0

                self.PWMServo_X = 0

                self.PWMServo_Y = -60

                self.s1_init_angle = Int32()

                self.s1_init_angle.data = self.PWMServo_X

                self.s2_init_angle = Int32()

                self.s2_init_angle.data = self.PWMServo_Y

                #create pub

                self.pub_goal = self.create_publisher(GoalID,"move_base/cancel",10)

                self.pub_cmdVel = self.create_publisher(Twist,'cmd_vel',10)

                self.pub_Buzzer = self.create_publisher(UInt16,"beep",  1)

                self.pub_JoyState = self.create_publisher(Bool,"JoyState",  10)

                self.pub_Servo1 = self.create_publisher(Int32,"servo_s1" , 10)

                self.pub_Servo2 = self.create_publisher(Int32,"servo_s2" , 10)




                #create sub

                self.sub_Joy = self.create_subscription(Joy,'joy', self.buttonCallback,1
0)


…（以下省略 174 行）
```

第 4 段｜手把遙控與搶方向盤 B4

▶ 看這段的精華影片（YouTube 私人影片，需登入擁有者帳號）

▼ 教練解說（6 句）

1. 第四段，遙控。鍵盤、手把、手機 App，三種方式最後都是發 cmd vel 這個話題。同一時間只能有一個在發，不然會搶方向盤。先開手把。
2. 手把已經連上了。我把速度限制在每秒 0.15 公尺、轉向 0.8 弧度，雲台上下這個功能也先關掉，因為剛才上下軸拉到線。請拿起手把，先按 R1 開啟控速。左搖桿前後是前進後退，右搖桿左右是旋轉，X 和 B 是雲台左右。Y 和 A 不要按。看右邊儀表板，玩三十秒，好了跟我說。
3. 我查到原因了，等一下解釋。現在請你：只按一次 R1，然後把左搖桿往前推住兩秒放開，再把右搖桿往左推兩秒放開。開始。
4. 原因是：輪子要先用 R1 打開控車開關，但剛才手把一直沒送出 R1，兩支搖桿前後和左右的訊號也沒收到，所以輪子沒有指令。我在左邊畫面整理了每個按鍵的功能，還有一個不能按的鍵。
5. 鍵盤這部分跳過，因為官方鍵盤程式一開始的速度就超過安全上限。現在把手把程式關掉，讓車停好。
6. 手把程式都關了，現在沒有人在發 cmd vel，車子靜止，雲台回正。這段學到：不管用哪種遙控，最後都是發同一個話題，而且官方程式的預設速度要先限制好才安全。第四段完成。

指令與輸出（13 個）

車上 ROS 2 容器 Inspect keyboard node for servo and speed defaults

```
$ f=/root/yahboomcar_ws/src/yahboomcar_ctrl/yahboomcar_ctrl/yahboom_keyboard.py; grep -nE 'servo|Int32|declare_parameter|speed|turn|moveBindings|speedBindings|^msg|q/z|w/x' $f | head -50
```

```
12:msg = """

20:q/z : increase/decrease max speeds by 10%

21:w/x : increase/decrease only linear speed by 10%

22:e/c : increase/decrease only angular speed by 10%

23:t/T : x and y speed switch

31:moveBindings = {

48:speedBindings = {
```

```
69:             self.declare_parameter("linear_speed_limit",1.0)

70:             self.declare_parameter("angular_speed_limit",5.0)

71:             self.linenar_speed_limit = self.get_parameter("linear_speed_limit").get_
parameter_value().double_value

72:             self.angular_speed_limit = self.get_parameter("angular_speed_limit").get
_parameter_value().double_value

80:             return key

81:     def vels(self, speed, turn):

82:             return "currently:\tspeed %s\tturn %s " % (speed,turn)

87:     xspeed_switch = True

88:     (speed, turn) = (0.2, 1.0)

96:             print(yahboom_keyboard.vels(speed, turn))

99:                     if key=="t" or key == "T": xspeed_switch = not xspeed_switch

103:                    if key in moveBindings.keys():

104:                            x = moveBindings[key][0]

105:                            th = moveBindings[key][1]

107:                    elif key in speedBindings.keys():

108:                            speed = speed * speedBindings[key][0]
…（以下省略 22 行）
```

容器內開視窗（Pi 桌面） Launch joy nodes with speed limits and s2 remapped

```
$ ros2 run joy joy_node
```

容器內開視窗（Pi 桌面） Launch joy nodes with speed limits and s2 remapped

```
$ ros2 run yahboomcar_ctrl yahboom_joy --ros-args -p xspeed_limit:=0.15 -p angular_speed_l
imit:=0.8 -r servo_s2:=/servo_s2_disabled
```

車上 ROS 2 容器 Launch joy nodes with speed limits and s2 remapped

```
$ sleep 4; ros2 node list
```

```
已在容器 5964fe81a259 背景啟動，畫面在 Pi 桌面（DISPLAY=:0）
log：/tmp/gui_20261027_050445.log（看 log：python ycar.py ros "cat /tmp/gui_20261027_050445.log"）
若畫面沒出現，可能要在 Pi 桌面先執行 xhost +local:
已在容器 5964fe81a259 背景啟動，畫面在 Pi 桌面（DISPLAY=:0）
log：/tmp/gui_20261027_050446.log（看 log：python ycar.py ros "cat /tmp/gui_20261027_050446.log"）
若畫面沒出現，可能要在 Pi 桌面先執行 xhost +local:
/YB_Car_Node
/joy_ctrl
/joy_node
/rqt_gui_cpp_node_2391
/ycar_monitor
---
Type: std_msgs/msg/Int32
Publisher count: 0
Subscription count: 2
Type: std_msgs/msg/Int32
Publisher count: 1
Subscription count: 0
```

車上 ROS 2 容器 Check joy node logs

```
$ cat /tmp/gui_20261027_050445.log | tail -5
```

```
[INFO] [1793048686.325466852] [joy_node]: No haptic (rumble) available, skipping initialization
[INFO] [1793048686.325637259] [joy_node]: Opened joystick: Controller.  deadzone: 0.050000
---
```

車上 ROS 2 容器 Check Joy_active toggle button and joy messages

```
$ sed -n '76,92p' /root/yahboomcar_ws/src/yahboomcar_ctrl/yahboomcar_ctrl/yahboom_joy.py
```

```
self.pub_Servo.publish(self.servo_angle)



        def user_jetson(self, joy_data):

                #cancel nav

                if joy_data.buttons[7] == 1: self.cancel_nav()

                #Buzzer

                if joy_data.buttons[11] == 1:
```

```
                        b = UInt16()

                        self.Buzzer_active = not self.Buzzer_active

                        b.data = self.Buzzer_active

                        self.pub_Buzzer.publish(b)









                xlinear_speed = self.filter_data(joy_data.axes[1]) * self.xspeed_limit *
self.linear_Gear

        #ylinear_speed = self.filter_data(joy_data.axes[2]) * self.yspeed_limit * self.l
inear_Gear

---
22:             self.Joy_active = False

105:            if self.Joy_active == True:

216:                    self.Joy_active = not self.Joy_active

217:                    Joy_ctrl.data = self.Joy_active

---
Type: geometry_msgs/msg/Twist
…（以下省略 34 行）
```

車上 ROS 2 容器 Read Joy_active toggle function

```
$ sed -n '203,225p' /root/yahboomcar_ws/src/yahboomcar_ctrl/yahboomcar_ctrl/yahboom_joy.py
```

```
twist.linear.y = ylinear_speed

                twist.angular.z = angular_speed

                for i in range(3):

                        self.pub_cmdVel.publish(twist)
```

```
        def filter_data(self, value):
                if abs(value) < 0.2: value = 0
                return value


        def cancel_nav(self):
                now_time = time.time()
                if now_time - self.cancel_time > 1:
                        Joy_ctrl = Bool()
                        self.Joy_active = not self.Joy_active
                        Joy_ctrl.data = self.Joy_active
                        for i in range(3):
                                self.pub_JoyState.publish(Joy_ctrl)
                                #self.pub_goal.publish(GoalID())
                                self.pub_cmdVel.publish(Twist())
                        self.cancel_time = now_time


def main():
        rclpy.init()
```

車上 ROS 2 容器 Capture live gamepad inputs for 15 s, then stop car

```
$ timeout 15 ros2 topic echo --csv /joy 2>/dev/null | cut -d, -f4- | awk '!seen[$0]++' | h
ead -40
```

停車 Capture live gamepad inputs for 15 s, then stop car

```
$ ycar.py stopcar


1.0,-0.0,-0.0,-0.0,1.0,1.0,0.0,0.0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
0.4468761086463928,-0.0,-0.0,-0.0,1.0,1.0,0.0,0.0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
```

```
-0.8596469759941101,-0.0,-0.0,-0.0,1.0,1.0,0.0,0.0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
-0.0,-0.0,-0.0,-0.0,1.0,1.0,0.0,0.0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
-0.0,-0.0,-0.0,-0.0,1.0,1.0,1.0,0.0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
0.3643155097961426,-0.0,-0.0,-0.0,1.0,1.0,0.0,0.0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
0.8514230251312256,-0.0,-0.0,-0.0,1.0,1.0,0.0,0.0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
-0.38079550862312317,-0.0,-0.0,-0.0,1.0,1.0,0.0,0.0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
-0.0,-0.0,-0.0,-0.0,1.0,1.0,0.0,1.0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
-0.0,-0.0,-0.0,-0.0,1.0,1.0,0.0,1.0,0,0,0,0,0,0,0,1,0,0,0,0,0,0
-0.0,-0.0,-0.0,-0.0,1.0,-1.0,0.0,1.0,0,0,0,0,0,0,0,1,0,0,0,0,0,0
-0.0,-0.0,-0.0,-0.0,1.0,-1.0,0.0,1.0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
-0.0,-0.0,-0.0,-0.0,1.0,1.0,0.0,0.0,0,0,0,0,0,0,0,0,1,0,0,0,0,0
-0.0,-0.0,-0.0,-0.0,-1.0,1.0,0.0,0.0,0,0,0,0,0,0,0,0,1,0,0,0,0,0
-0.0,-0.0,-0.0,-0.0,-0.6202212572097778,1.0,0.0,0.0,0,0,0,0,0,0,0,0,1,0,0,0,0,0
-0.0,-0.0,-0.0,-0.0,-0.08357734233140945,1.0,0.0,0.0,0,0,0,0,0,0,0,0,1,0,0,0,0,0
-0.0,-0.0,-0.0,-0.0,-0.08357734233140945,1.0,0.0,0.0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
===JoyState
(這 2 秒沒有新訊息)
/cmd_vel 訂閱者數量：1
已送出零速度 /cmd_vel 第 1 次
已送出零速度 /cmd_vel 第 2 次
已送出零速度 /cmd_vel 第 3 次
已送出 /servo_s1=0、/servo_s2=0、/beep=0
```

結束容器內程式 Kill gamepad nodes, stop car, verify

```
$ ycar.py kill yahboom_joy
```

結束容器內程式 Kill gamepad nodes, stop car, verify

```
$ ycar.py kill joy_node
```

停車 Kill gamepad nodes, stop car, verify

```
$ ycar.py stopcar
```

車上 ROS 2 容器 Kill gamepad nodes, stop car, verify

```
$ ros2 node list
```

```
bash killed (pid 5536)
ros2 killed (pid 5600)
yahboom_joy killed (pid 5601)
bash killed (pid 5458)
ros2 killed (pid 5523)
joy_node killed (pid 5524)
/cmd_vel 訂閱者數量：1
已送出零速度 /cmd_vel 第 1 次
```

```
已送出零速度 /cmd_vel 第 2 次
已送出零速度 /cmd_vel 第 3 次
已送出 /servo_s1=0、/servo_s2=0、/beep=0
/YB_Car_Node
/rqt_gui_cpp_node_2391
/ycar_monitor
---
Publisher count: 0
```

插曲｜相機 USB 斷線排查 硬體除錯

▼ 教練解說（2 句）

1. 好，把雲台慢慢往下轉到負 45 度，每次五度，左右保持在中間。
2. 相機還沒正常。系統有看到它插上，但連線品質不好，只跑在低速模式，初始化失敗，所以沒有產生影像裝置。第五段先不開始，請看左邊畫面的說明。

指令與輸出（4 個）

車上 ROS 2 容器 Tilt servo down to -45 in 5-degree steps

```
$ for a in $(seq -5 -5 -45); do ros2 topic pub --once /servo_s2 std_msgs/msg/Int32 "{data: $a}" >/dev/null
```

```
-5 -10 -15 -20 -25 -30 -35 -40 -45
```

Pi 5 主機 Check camera on host and in container

```
$ lsusb | grep -iE '0c45|camera'
```

車上 ROS 2 容器 Check camera on host and in container

```
$ ls -l /dev/video0
```

```
Bus 001 Device 011: ID 0c45:6368 Microdia USB 2.0 Camera
ls: cannot access '/dev/video0': No such file or directory
---
c----------+ 0 root root 81, 19 Oct 15 14:17 /dev/video0
   2391       33:00 python3 /opt/ros/humble/lib/rqt_image_view/rqt_image_view
```

Pi 5 主機 Find camera device number and recent kernel log

```
$ ls -l /dev/video[0-9] /dev/video1[0-8]
```

```
ls: cannot access '/dev/video[0-9]': No such file or directory
ls: cannot access '/dev/video1[0-8]': No such file or directory
ls: cannot access '/dev/v4l/by-id/': No such file or directory
[ 4243.025583] usb 1-1: Device not responding to setup address.
[ 4243.233496] usb 1-1: device not accepting address 10, error -71
[ 4243.361491] usb 1-1: new full-speed USB device number 11 using xhci-hcd
[ 4243.394739] usb 1-1: not running at top speed; connect to a high speed hub
```

```
[ 4243.489746] usb 1-1: New USB device found, idVendor=0c45, idProduct=6368, bcdDevice=27.
02
[ 4243.489752] usb 1-1: New USB device strings: Mfr=2, Product=1, SerialNumber=0
[ 4243.489755] usb 1-1: Product: USB 2.0 Camera
[ 4243.489758] usb 1-1: Manufacturer: Sonix Technology Co., Ltd.
[ 4243.511786] usb 1-1: Found UVC 1.00 device USB 2.0 Camera (0c45:6368)
[ 4243.627839] uvcvideo 1-1:1.1: Failed to set UVC probe control : -71 (exp. 26).
[ 4243.629846] uvcvideo 1-1:1.1: Failed to query (129) UVC probe control : -71 (exp. 26).
[ 4243.629851] uvcvideo 1-1:1.1: Failed to initialize the device (-71).
```

第 5 段｜OpenCV 與影像話題 M1

▶ 看這段的精華影片（YouTube 私人影片，需登入擁有者帳號）

▼ 教練解說（6 句）

1. 第五段，OpenCV 和影像話題。先跑一個不用相機的範例：讀一張圖，轉成灰階、模糊，再找出邊緣。請看螢幕三。
2. 螢幕三應該有三個視窗：原圖、灰階、還有黑底白線的邊緣圖。灰階是把三個顏色合成一個亮度；模糊是先去掉雜點；Canny 找亮度突然變化的地方，就是物體的邊緣。
3. 範例關掉了。接下來換真的相機，先確認相機裝置在，再開相機節點。
4. 相機節點開好了，每秒大約三十張，發在 image raw 這個話題，型別是 sensor msgs 的 Image。現在開影像視窗，把話題的畫面顯示在螢幕三。
5. 螢幕三應該看到相機的即時畫面了。這裡的重點是 cv bridge：ROS 傳的是 Image 訊息，OpenCV 要的是陣列，cv bridge 就是在兩者之間翻譯。我們自己寫影像程式時，就是訂閱 image raw，用 cv bridge 轉成 OpenCV 圖片再處理。
6. 這段開的程式都關了，相機現在沒有人占用，下一段的手部偵測可以直接用。第五段完成。

指令與輸出（12 個）

容器內開視窗（Pi 桌面） Narrate and launch OpenCV Canny demo

```
$ cd /root/yahboomcar_ws/src/yahboomcar_astra/scripts/opencv && python3 3_3.py
```

```
已在容器 5964fe81a259 背景啟動，畫面在 Pi 桌面（DISPLAY=:0）
log：/tmp/gui_20261027_052312.log（看 log：python ycar.py ros "cat /tmp/gui_20261027_05231
2.log"）
若畫面沒出現，可能要在 Pi 桌面先執行 xhost +local:
```

車上 ROS 2 容器 Explain Canny and check process running

```
$ cat /tmp/gui_20261027_052312.log | tail -5; ps -eo pid,cmd | grep '[3]_3.py'
```

```
   6579 bash -c source /opt/ros/humble/setup.bash; export ROS_DOMAIN_ID=20; if [ -f /root/
yahboomcar_ws/install/setup.bash ]; then source /root/yahboomcar_ws/install/setup.bash; f
i; cd /root/yahboomcar_ws/src/yahboomcar_astra/scripts/opencv && python3 3_3.py > /tmp/gui
_20261027_052312.log 2>&1
   6644 python3 3_3.py
```

結束容器內程式 Kill demo, check video0, start usb_cam

```
$ ycar.py kill 3_3.py
```

車上 ROS 2 容器 Kill demo, check video0, start usb_cam

```
$ ls -l /dev/video0
```

容器內開視窗（Pi 桌面） Kill demo, check video0, start usb_cam

```
$ ros2 run usb_cam usb_cam_node_exe
```

```
bash killed (pid 6579)
python3 killed (pid 6644)


c---------+ 0 root root 81, 19 Oct 15 14:17 /dev/video0
已在容器 5964fe81a259 背景啟動，畫面在 Pi 桌面（DISPLAY=:0）
log：/tmp/gui_20261027_052354.log（看 log：python ycar.py ros "cat /tmp/gui_20261027_05235
4.log"）
若畫面沒出現，可能要在 Pi 桌面先執行 xhost +local:
```

車上 ROS 2 容器 Check usb_cam log and image topic rate

```
$ sleep 3; tail -8 /tmp/gui_20261027_052354.log
```

```
[INFO] [1793049836.228168423] [usb_cam]: Setting 'white_balance_temperature_auto' to 1
[INFO] [1793049836.228223221] [usb_cam]: Setting 'exposure_auto' to 3
unknown control 'exposure_auto'

[INFO] [1793049836.232117527] [usb_cam]: Setting 'focus_auto' to 0
unknown control 'focus_auto'

[INFO] [1793049836.322540966] [usb_cam]: Timer triggering every 33 ms
---
Type: sensor_msgs/msg/Image
Publisher count: 1
Subscription count: 1
average rate: 29.782
        min: 0.031s max: 0.037s std dev: 0.00204s window: 91
```

容器內開視窗（Pi 桌面） Launch rqt_image_view on image_raw

```
$ ros2 run rqt_image_view rqt_image_view /image_raw
```

```
已在容器 5964fe81a259 背景啟動，畫面在 Pi 桌面（DISPLAY=:0）
```

```
log：/tmp/gui_20261027_052429.log（看 log：python ycar.py ros "cat /tmp/gui_20261027_052429.log"）
若畫面沒出現，可能要在 Pi 桌面先執行 xhost +local:
```

車上 ROS 2 容器 Explain cv_bridge and show subscribers

```
$ ros2 topic info /image_raw
```

```
Type: sensor_msgs/msg/Image
Publisher count: 1
Subscription count: 2
```

車上 ROS 2 容器 Close segment-5 camera programs and verify

```
$ for p in $(pgrep -f rqt_image_view); do [ "$p" != 2391 ] && kill $p
```

結束容器內程式 Close segment-5 camera programs and verify

```
$ ycar.py kill usb_cam_node_exe
```

車上 ROS 2 容器 Close segment-5 camera programs and verify

```
$ sleep 2; ps -eo pid,cmd | grep -E '[r]qt_image_view|[u]sb_cam|[3]_3.py'; ros2 topic info /image_raw | grep Publisher
```

```
killed 6968
killed 7033
killed 7034
killed 7125
bash killed (pid 6794)
ros2 killed (pid 6859)
usb_cam_node_ex killed (pid 6860)
   2391 python3 /opt/ros/humble/lib/rqt_image_view/rqt_image_view
   7034 python3 /opt/ros/humble/lib/rqt_image_view/rqt_image_view /image_raw
Publisher count: 0
```

車上 ROS 2 容器 Force-close remaining rqt_image_view of this segment

```
$ kill -INT 7034 2>/dev/null
```

```
2391 python3 /opt/ros/humble/lib/rqt_image_view/rqt_image_view
```

第 6 段｜MediaPipe 手部與臉部偵測 M2

▶ 看這段的精華影片（YouTube 私人影片，需登入擁有者帳號）

▼ 教練解說（5 句）

1. 第六段，用 MediaPipe 做手部偵測。這個程式直接讀相機，不會動到輪子和雲台。我現在把它打開，畫面會出現在螢幕三。
2. 手部偵測開好了。雲台現在朝下，請把手放到車上鏡頭前面，看螢幕三，依序比一、比二、張開五指、再握拳。MediaPipe 會在每隻手標出 21 個關鍵點：手腕一個，每根手指四個。比完跟我說好了。
3. 很好。手部偵測先關掉，把相機讓給下一個程式。
4. 接著試臉部網格 FaceMesh，它會在臉上標出大約 468 個點。雲台朝下不能抬，所以請你把臉低下來，靠近鏡頭正前方。我開十五秒。
5. 十五秒到了，FaceMesh 關掉了。這段我們用同一顆相機做了手部 21 點和臉部網格，原理都是先找到手或臉，再用模型估計關鍵點的位置。第六段完成。

指令與輸出（12 個）

車上 ROS 2 容器 Check mediapipe scripts for actuator publishers

```
$ d=/root/yahboomcar_ws/src/yahboomcar_mediapipe; f1=$(find $d -name '01_HandDetector*.py' | grep -v build | head -1); f2=$(find $d -name '04_FaceMesh*.py' | grep -v build | head -1)
```

```
/root/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/scripts/01_HandDetector.py /root/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/scripts/04_FaceMesh.py
/root/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/scripts/01_HandDetector.py:66:    capture = cv.VideoCapture(0)
/root/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/scripts/04_FaceMesh.py:60:    capture = cv.VideoCapture(0)
c---------+ 0 root root 81, 19 Oct 15 14:17 /dev/video0
相機沒被占用
```

容器內開視窗（Pi 桌面） Launch hand detector and check it runs

```
$ ros2 run yahboomcar_mediapipe 01_HandDetector
```

車上 ROS 2 容器 Launch hand detector and check it runs

```
$ sleep 6; tail -5 $(ls -t /tmp/gui_*.log | head -1); ps -eo pid,cmd | grep '[0]1_HandDete
ctor' | head -2
```

```
已在容器 5964fe81a259 背景啟動，畫面在 Pi 桌面（DISPLAY=:0）
log：/tmp/gui_20261027_052640.log（看 log：python ycar.py ros "cat /tmp/gui_20261027_05264
0.log"）
若畫面沒出現，可能要在 Pi 桌面先執行 xhost +local:
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
   7421 bash -c source /opt/ros/humble/setup.bash; export ROS_DOMAIN_ID=20; if [ -f /root/
yahboomcar_ws/install/setup.bash ]; then source /root/yahboomcar_ws/install/setup.bash; f
i; ros2 run yahboomcar_mediapipe 01_HandDetector > /tmp/gui_20261027_052640.log 2>&1
   7486 /usr/bin/python3 /opt/ros/humble/bin/ros2 run yahboomcar_mediapipe 01_HandDetector
```

結束容器內程式 Close hand detector and verify

```
$ ycar.py kill 01_HandDetector
```

車上 ROS 2 容器 Close hand detector and verify

```
$ sleep 2; ps -eo pid,cmd | grep '[0]1_HandDetector'
```

```
bash killed (pid 7421)
ros2 killed (pid 7486)
01_HandDetector killed (pid 7487)
已關閉
```

容器內開視窗（Pi 桌面） Launch FaceMesh demo

```
$ ros2 run yahboomcar_mediapipe 04_FaceMesh
```

```
已在容器 5964fe81a259 背景啟動，畫面在 Pi 桌面（DISPLAY=:0）
log：/tmp/gui_20261027_052843.log（看 log：python ycar.py ros "cat /tmp/gui_20261027_05284
3.log"）
若畫面沒出現，可能要在 Pi 桌面先執行 xhost +local:
```

車上 ROS 2 容器 Wait 15 s, close FaceMesh, verify

```
$ sleep 15; tail -3 /tmp/gui_20261027_052843.log
```

結束容器內程式 Wait 15 s, close FaceMesh, verify

```
$ ycar.py kill 04_FaceMesh
```

車上 ROS 2 容器 Wait 15 s, close FaceMesh, verify

```
$ sleep 2; ps -eo pid,cmd | grep -E '[0]4_FaceMesh|[0]1_HandDetector'
```

```
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
bash killed (pid 7672)
ros2 killed (pid 7737)
04_FaceMesh killed (pid 7738)
都已關閉
```

車上 ROS 2 容器 Inspect colorTracker servo/cmd_vel behavior

```
$ f=$(find /root/yahboomcar_ws/src/yahboomcar_astra -name 'colorTracker*.py' | grep -v bui
ld | head -1)
```

```
/root/yahboomcar_ws/src/yahboomcar_astra/launch/colorTracker_X3.launch.py
```

車上 ROS 2 容器 Find colorTracker source and its publishers

```
$ grep -nA3 colorTracker /root/yahboomcar_ws/src/yahboomcar_astra/setup.py; f=$(grep -rl
'class.*Tracker\|def main' /root/yahboomcar_ws/src/yahboomcar_astra/yahboomcar_astra/ | gr
ep -i track | head -3)
```

```
26:        'colorTracker = yahboomcar_astra.colorTracker:main',
27-        'qrTracker = yahboomcar_astra.qrTracker:main',
28-        'mono_Tracker = yahboomcar_astra.mono_Tracker:main',
29-        'face_fllow = yahboomcar_astra.face_fllow:main',
grep: /root/yahboomcar_ws/src/yahboomcar_astra/yahboomcar_astra/.colorTracker.py.swp: bi
nary file matches
grep: /root/yahboomcar_ws/src/yahboomcar_astra/yahboomcar_astra/.colorTracker.py.swn: bi
nary file matches
/root/yahboomcar_ws/src/yahboomcar_astra/yahboomcar_astra/.colorTracker.py.swp /root/yah
boomcar_ws/src/yahboomcar_astra/yahboomcar_astra/.colorTracker.py.swn /root/yahboomcar_w
s/src/yahboomcar_astra/yahboomcar_astra/mono_Tracker.py
== /root/yahboomcar_ws/src/yahboomcar_astra/yahboomcar_astra/.colorTracker.py.swp
== /root/yahboomcar_ws/src/yahboomcar_astra/yahboomcar_astra/.colorTracker.py.swn
== /root/yahboomcar_ws/src/yahboomcar_astra/yahboomcar_astra/mono_Tracker.py
13:        self.pub_Servo1 = self.create_publisher(Int32,"servo_s1" , 10)
14:        self.pub_Servo2 = self.create_publisher(Int32,"servo_s2" , 10)
16:        self.target_servox = 0
17:        self.target_servoy = -60
52:        self.declare_parameter("Kp",12)
54:        self.declare_parameter("Ki",0)
56:        self.declare_parameter("Kd",0.9)
92:        self.target_servox = 0
93:        self.target_servoy = -60
100:            self.target_servox -= x_Pid
101:            self.target_servoy += y_Pid
```

```
103:            self.target_servox -= x_Pid
104:            self.target_servoy += y_Pid
105:        if self.target_servox >= 90:
106:            self.target_servox = 90
107:        elif self.target_servox <= -90:
108:            self.target_servox = -90
109:        if self.target_servoy >= 20:
110:            self.target_servoy = 20
111:        elif self.target_servoy <= -90:
112:            self.target_servoy = -90
113:        print("servo1",self.target_servox)
114:        servo1_angle = Int32()
115:        servo1_angle.data = int(self.target_servox)
116:        servo2_angle = Int32()
117:        servo2_angle.data = int(self.target_servoy)
118:        self.pub_Servo1.publish(servo1_angle)
119:        self.pub_Servo2.publish(servo2_angle)
```

車上 ROS 2 容器 Grep colorTracker.py publishers and servo logic

```
$ f=/root/yahboomcar_ws/src/yahboomcar_astra/yahboomcar_astra/colorTracker.py; grep -nE 'i
mport|create_publisher|create_subscription|servo|target_servo|Twist|cmd_vel' $f | head -40
```

```
2:import rclpy
3:from rclpy.node import Node
4:from std_msgs.msg import Bool
5:from geometry_msgs.msg import Twist
6:from sensor_msgs.msg import CompressedImage, LaserScan, Image
7:from yahboomcar_msgs.msg import Position
8:from std_msgs.msg import Int32, Bool,UInt16
10:import os
11:import threading
12:import math
13:import time
14:from yahboomcar_astra.astra_common import *
15:from yahboomcar_msgs.msg import Position
16:from cv_bridge import CvBridge
17:print("import done")
23:        self.pub_cmdVel = self.create_publisher(Twist,'/cmd_vel',10)
24:        self.pub_Servo1 = self.create_publisher(Int32,"servo_s1" , 10)
25:        self.pub_Servo2 = self.create_publisher(Int32,"servo_s2" , 10)
27:        self.sub_depth = self.create_subscription(Image,"/image_raw", self.depth_img_
Callback, 1)
28:        self.sub_JoyState = self.create_subscription(Bool,'/JoyState',  self.JoyState
Callback,1)
29:        self.sub_position = self.create_subscription(Position,"/Current_point",self.p
ositionCallback,1)
135:                self.pub_cmdVel.publish(Twist())
142:        self.pub_cmdVel.publish(Twist())
```

```
151:         self.pub_cmdVel.publish(Twist())
173:         # rospy.loginfo("target_servox: {}, target_servoy: {}".format(self.target_se
rvox, self.target_servoy))
174:         print("servo1",self.PWMServo_X)
175:         servo1_angle = Int32()
176:         servo1_angle.data = int(self.PWMServo_X)
177:         servo2_angle = Int32()
178:         servo2_angle.data = int(self.PWMServo_Y)
179:         self.pub_Servo1.publish(servo1_angle)
180:         self.pub_Servo2.publish(servo2_angle)
```

第 7 段｜自己寫的避障節點上真車 M5

▶ 看這段的精華影片（YouTube 私人影片，需登入擁有者帳號）

▼ 教練解說（4 句）

1. 第八段，自己的避障節點。這是講義 M5 的程式，之前在電腦模擬測過，現在搬到真車上：前方 0.3 公尺內有東西就停，沒有就慢慢前進。先看關鍵的幾行。
2. 程式訂閱光達的 scan 話題，算出正前方最近的距離；小於 0.3 公尺就發零速度，否則發每秒 0.1 公尺前進，都發到 cmd vel。接下來執行三十秒，輪子會轉。請把手放在車子正前方二十公分，看輪子停；手拿開，輪子又轉。試完跟我說好了。
3. 三十秒結束，避障程式已關閉，車子靜止，雲台保持朝下。紀錄顯示你擋了五次，每次距離在 0.22 到 0.27 公尺就停車，手拿開就繼續前進，跟模擬的結果一樣。
4. 總結今天在真車上做的事：確認連線和硬體，看懂 agent 和容器，讀了電池、光達、IMU，讓輪子、雲台、蜂鳴器動起來；手把轉了雲台；跑了 OpenCV 和相機話題、手部和臉部偵測，最後讓自己寫的避障節點在真車上運作。手把開車和顏色追蹤這次沒做成，留到下次。下一次要把車放到地上，做校準、避障、建圖和導航。第八段完成，第一階段演練全部完成。

指令與輸出（5 個）

容器內執行 Python 檔 Run obstacle_stop 30 s, then send zero velocity

```
$ ycar.py rospy obstacle_stop.py
```

車上 ROS 2 容器 Run obstacle_stop 30 s, then send zero velocity

```
$ timeout 2 ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist '{}' >/dev/null
```

車上 ROS 2 容器 Identify remaining cmd_vel publisher

```
$ ps -eo pid,etime,cmd | grep -E '[o]bstacle_stop|[t]opic pub' ; ros2 topic info -v /cmd_v
el | grep -E 'Node name|Endpoint type'; timeout 3 ros2 topic echo /cmd_vel --once 2>/dev/n
ull
```

```
Node name: obstacle_stop
Node namespace: /
Endpoint type: PUBLISHER
Node name: YB_Car_Node
Node namespace: /
Endpoint type: SUBSCRIPTION
Node name: ycar_monitor
```

```
Node namespace: /
Endpoint type: SUBSCRIPTION
(3 秒內沒有新的 cmd_vel)
```

結束容器內程式 Ensure obstacle_stop killed, send zero, list nodes

```
$ ycar.py kill obstacle_stop
```

車上 ROS 2 容器 Ensure obstacle_stop killed, send zero, list nodes

```
$ timeout 2 ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist '{}' >/dev/null
```

```
容器內沒有符合「obstacle_stop」的程序
/YB_Car_Node
/rqt_gui_cpp_node_2391
/ycar_monitor
```

← 回學習地圖